Skill Cap - Milestone 3

Integration, Distribution & Deployment | Daniel Khant | CMPE 172 | October 27, 2026

Overview and distribution design

Skill Cap is a game coaching booking system built with Spring Boot, Thymeleaf, JDBC, and PostgreSQL. This milestone adds booking and cancellation confirmations, fault handling, health checks, metrics, Docker, and CI. The mock notification service stores requests; it does not send real email or SMS.

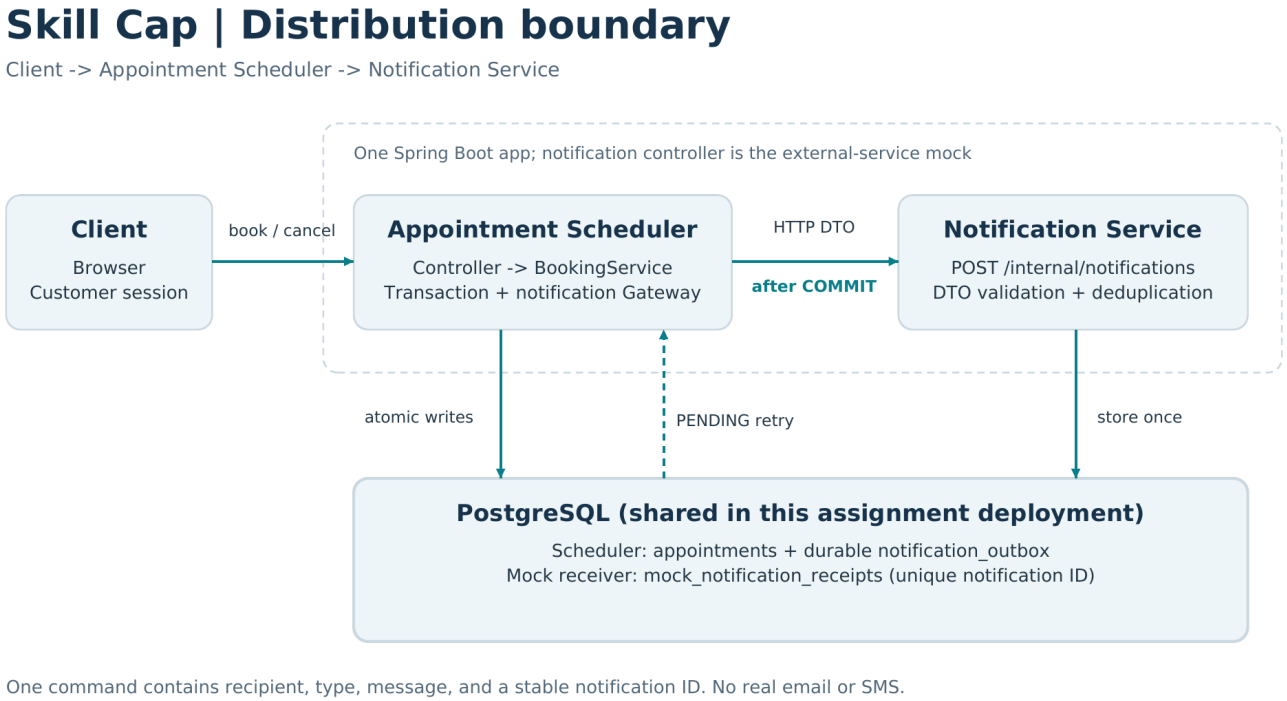


Figure 1. The Gateway calls a second Spring controller over HTTP after the booking transaction commits. Both controllers share the assignment app and PostgreSQL deployment.

Coarse-grained interface

One POST sends a complete notification command. The receiver needs no additional scheduler calls or database entities. This keeps the boundary small and lets a real channel replace the mock without changing booking rules.

```
public record NotificationRequest(
        String notificationId, String recipient,
        NotificationType type, String message) { }
```

The actual record also validates required fields, recipient format, ID format, and maximum lengths. Types are BOOKING_CONFIRMATION and CANCELLATION_CONFIRMATION. The mock requires the shared X-Notification-Token header.

AI tools used: ChatGPT assisted with the integration code, tests, documentation, and this report. The recorded validation describes the checks actually executed.

Transactions and fault handling

Commit first, then notify

BookingService runs without a caller transaction and invokes a separate proxied BookingTransactionService. The transaction locks the slot, creates the appointment, and inserts a PENDING outbox event. The method returns only after commit. The dispatcher then performs HTTP without holding the booking transaction or slot lock. Cancellation updates ownership-checked appointment state and queues its own event in the same way.

```
// BookingService.book: the transaction has committed on return.
request = transactions.book(userId, slotId);
metrics.bookingSucceeded();
dispatcher.dispatch(request);

// BookingTransactionService: called within its DB transaction.
NotificationRequest request =
        appointmentRepository.notificationFor(appointmentId, type);
outbox.enqueue(request);
return request;
```

If the outbox insert fails, the appointment change rolls back too. If HTTP fails after commit, the appointment remains valid and the event stays pending. A scheduled worker retries up to 25 pending events per pass, with a 30-second delay between passes.

Timeout, retry, and fallback

Setting	Default / behavior
Connection / read timeout	500 ms / 1,000 ms per attempt
Immediate attempts	At most 3; delays of 100 ms then 200 ms
Retriable failures	Network/timeout errors, HTTP 5xx, and HTTP 429
Other HTTP 4xx	Stop immediate retries; retain pending event for correction/recovery
Fallback	Keep PENDING event in PostgreSQL and return booking/cancellation success

```
client.post().uri("/internal/notifications")
    .header("X-Notification-Token", properties.token())
    .body(request).retrieve().toBodilessEntity();
```

Idempotency and recovery

All attempts reuse a stable appointment-and-event ID and the same payload. The receiver stores that ID as a primary key. Identical duplicates return success; a changed payload with the same ID returns 409. If a response is lost after acceptance, replaying the pending event does not create another receipt. Outbox and receipt state survive restarts.

Failure scenario: the notification endpoint is unavailable. Booking still confirms, delivery is logged as deferred, and the pending event persists. After restoring the endpoint, the worker delivers it without rebooking. Delivery can be delayed or reordered; this is at-least-once retry with idempotent mock acceptance, not a claim of exactly-once real email.

System management

Logging plan

Logs identify operations using slot, appointment, and notification IDs. Application log messages omit passwords, shared tokens, recipient addresses, and message text. Invalid notification payloads receive a sanitized response/log entry.

Level	What is recorded
INFO	Committed booking/cancellation; mock acceptance and delivery completion
WARN	Rejected booking, failed HTTP attempt, and durable deferral
ERROR	Database-health failure, unexpected dispatch failure, or failed recovery scan

Examples emitted by the code (IDs vary by operation):

```
INFO  Booking committed slotId=1
      notificationId=appointment-1-booking-confirmation
WARN  Notification deferred
      notificationId=appointment-1-booking-confirmation
      recovery=outbox-retry
ERROR Database health check failed reason=SQLException
```

Health endpoint

GET /health obtains a database connection and calls isValid(2). Success returns HTTP 200 with `{"status":"UP","database":"UP"}`. Failure returns HTTP 503 with both fields DOWN. Pool acquisition waits at most three seconds; validation uses a two-second timeout. Notification availability is excluded because the scheduler can continue accepting bookings during its outage.

```
try (Connection connection = dataSource.getConnection()) {
    if (connection.isValid(2)) {
        return ResponseEntity.ok(
                Map.of("status", "UP", "database", "UP"));
    }
}
```

Metrics endpoint

GET /metrics returns process-lifetime JSON counters and average booking service latency. Counters are thread-safe and stay available independently of database connectivity.

Metric	Meaning
bookings_total / failed_bookings_total	Successful committed bookings / rejected or failed booking calls
cancellations_total	Successful committed cancellations
average_booking_latency_ms	Average service-call duration, including notification waits and failed calls
notification_http_attempts_total	Outbound HTTP attempts, including retries
notification_delivery_completions_total / notification_deferrals_total	Delivery completions and deferral occurrences; replays can increment these again

These counters reset on restart and are per app instance. They are not an hourly aggregate or Prometheus endpoint. Persistent operational evidence is in the appointment history and outbox/receipt tables.

Delivery, testing, and results

Packaging and CI

The Dockerfile builds and tests the Java 17 application, then copies its executable JAR into a Java runtime image running as a non-root user. A Java health probe calls /health. Compose starts the app and PostgreSQL 16, waits for database readiness, and retains data in a named volume. Local .env settings are ignored by Git and Docker builds.

```
# .github/workflows/ci.yml
on:
  push:
  pull_request:
# A disposable PostgreSQL service is supplied to this job.
- name: Build and run H2 plus PostgreSQL tests
  run: bash ./mvnw -B clean verify
```

The workflow also validates Compose and builds the Docker image. The notification mock stays in the app, so Compose needs two containers: the app and PostgreSQL.

Executed validation

Maven clean verify: BUILD SUCCESS. 26 tests discovered; 24 passed; 0 failures; 0 errors; 2 skipped. The skipped tests require a disposable PostgreSQL server and are enabled by the CI environment. The executable Spring Boot JAR was built successfully.

Check	Result
HTTP booking/cancellation with notification endpoint down	Passed; confirmation redirect and database state verified
Commit visibility and released slot lock before HTTP	Passed using a second database connection
Outbox failure rollback; recovery; duplicate replay	Passed
Slow responses, retries, DTO/token validation	Passed
Health/metrics and original booking race/RBAC tests	Passed
POM, Compose, and workflow structure	Parsed and checked
PostgreSQL-specific runtime tests	2 skipped locally; configured for PostgreSQL 16 in CI
Docker startup / actual GitHub Actions run	Not executed in this workspace

Required outage integration test

```
var response = post(http, "/customer/appointments", "slotId=1");
assertEquals(302, response.statusCode());
assertEquals("BOOKED", jdbc.queryForObject(
        "SELECT status FROM appointments", String.class));
assertEquals(1, pending());
```

NotificationOutageIntegrationTest starts the real web app, logs in over HTTP, and uses an unreachable notification URL. It also checks cancellation and rebooking. The H2 schema models the BOOKED-only unique constraint with a generated key; CI PostgreSQL tests use the production partial index. Full results are in docs/maven-clean-verify.txt.

Before submission

Run Docker Desktop/Compose, push the updated source and check Actions, and record a code walkthrough of at least five minutes. Repository: github.com/DenialCant/cmpe172-game-coaching-booking. Add the recorded video link and any final runtime evidence to the editable report before submission. No Docker/CI screenshots or cloud deployment results are claimed here.